

HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop (HITL)

Prepared by: Kirthika R S

Internship Project — 2025

1. System Overview

1.1 Problem Definition

Traditional customer support systems rely on human agents to answer every query, which leads to high operational costs, long wait times, and inconsistent responses. There is a need for an intelligent, automated system that can answer customer queries accurately using a structured knowledge base.

This project solves this problem by building a RAG-Based Customer Support Assistant — a system that reads a PDF knowledge base, retrieves relevant information using vector search, and generates accurate, context-aware answers using a Large Language Model (LLM).

1.2 Scope of the System

- Ingest a PDF document as the knowledge base
- Split and embed the document into a vector store (FAISS)
- Accept natural language queries from users via CLI
- Retrieve relevant chunks and generate answers using Groq LLM
- Use LangGraph for workflow orchestration with conditional routing
- Escalate to a human agent when the bot cannot answer confidently

2. Architecture Diagram

The system follows a pipeline architecture with the following high-level flow:

Layer	Component	Description
User Interface	CLI (Terminal)	User types queries and receives answers
Ingestion Pipeline	PDF Loader + Chunker	Reads PDF and splits into chunks
Embedding System	HuggingFace Embeddings	Converts text chunks to vectors
Vector Database	FAISS	Stores and retrieves embeddings
Retrieval Layer	LangChain Retriever	Finds top-k relevant chunks
LLM Layer	Groq (LLaMA 3.3)	Generates answers from context
Workflow Engine	LangGraph	Controls flow between nodes
HITL System	Human Escalation Module	Handles unresolvable queries

Flow Diagram (Text Representation):

User Query → Process Node → [Condition] → Answer Node → Output

↓
Escalate Node → Human Agent → Output

2. Component Description

3.1 Document Loader

Tool: PyPDFLoader (LangChain Community)

- Reads and parses the PDF knowledge base file
- Extracts raw text page by page
- Returns LangChain Document objects for further processing

3.2 Chunking Strategy

Tool: RecursiveCharacterTextSplitter

- Chunk size: 500 characters
- Chunk overlap: 50 characters to preserve context across chunks
- Recursive splitting ensures natural text boundaries are respected

3.3 Embedding Model

Model: all-MiniLM-L6-v2 (HuggingFace)

- Lightweight and fast sentence transformer model
- Converts text chunks into 384-dimensional vectors
- Free to use — no API key required

3.4 Vector Store

Tool: FAISS (Facebook AI Similarity Search)

- Stores all chunk embeddings locally
- Supports fast similarity search using L2 distance
- Saved to disk for reuse without re-ingestion

3.5 Retriever

Retrieves top-3 most relevant chunks for a given query using cosine similarity search via FAISS.

3.6 LLM

Model: LLaMA 3.3 70B via Groq API

- Fast inference using Groq hardware
- Free tier available
- Receives retrieved context + user query and generates a final answer

3.7 Graph Workflow Engine

Tool: LangGraph

- Manages the flow between Process Node, Answer Node, and Escalate Node
- Uses a typed State object to pass data between nodes
- Supports conditional routing based on retrieval confidence

3.8 Routing Layer

- If retrieved context length is sufficient → route to Answer Node
- If context is missing or too short → route to Escalate Node

3.9 HITL Module

- Triggered when bot confidence is low
- Prompts a human agent to type a manual response
- Human response is returned as the final answer to the user

4. Data Flow

4.1 Ingestion Flow

PDF File → PyPDFLoader → Text Chunks → HuggingFace Embeddings → FAISS Vector Store → Saved to Disk

4.2 Query Lifecycle

1. User enters a query via CLI
2. LangGraph Process Node is triggered
3. Query is embedded and searched in FAISS
4. Top-3 relevant chunks are retrieved
5. If context is sufficient → Answer Node generates response via Groq LLM
6. If context is insufficient → Escalate Node requests human input
7. Final answer is displayed to the user

5. Technology Choices

Technology	Purpose	Reason for Choice
FAISS	Vector Store	Lightweight, no server needed, easy local setup
LangGraph	Workflow Engine	Graph-based control flow, supports HITL natively
Groq LLaMA 3.3	LLM	Free API, very fast inference, high quality
HuggingFace Embeddings	Embedding Model	Free, no API key, works offline
LangChain	Orchestration	Simplifies PDF loading, chunking and retrieval
Python 3.9	Language	Widely supported, rich AI/ML ecosystem

6. Scalability Considerations

6.1 Handling Large Documents

- Chunk size and overlap can be tuned for larger PDFs
- Multiple PDFs can be loaded and merged into a single vector store
- FAISS supports millions of vectors efficiently

6.2 Increasing Query Load

- FAISS retrieval is near real-time even at scale
- Groq API supports concurrent requests
- LangGraph workflows are stateless and can run in parallel

6.3 Latency Concerns

- Embedding is done at ingestion time, not query time
- Groq LLM provides sub-second response times
- FAISS local search eliminates network latency for retrieval

--- End of HLD Document ---